

HashMap and HashSet Solutions

1. Count frequency of characters/words using HashMap.

Simple Explanation: We use a **HashMap** where the key is the character or word, and the value is the count. We loop through the input; if the key already exists, we increase its count. If it doesn't exist, we add it with a count of 1.

```
import java.util.HashMap;
import java.util.Map;

public class FrequencyCounter {
    public static Map<String, Integer> countWordFrequency(String text) {
        Map<String, Integer> wordCounts = new HashMap<>();

        // 1. Normalize and split the text into words
        String[] words = text.toLowerCase().split("\\s+");

        // 2. Count frequency
        for (String word : words) {
            // Using getOrDefault to simplify incrementing the count
            wordCounts.put(word, wordCounts.getOrDefault(word, 0) + 1);
        }
        return wordCounts;
    }
}
```

2. Detect duplicates in array using HashSet.

Simple Explanation: A **HashSet** only stores unique elements. We loop through the array and try to add each element to the set. If the `add()` method returns **false**, it means the element is already present, and we have found a duplicate.

```
import java.util.HashSet;
import java.util.Set;

public class DuplicateDetector {
    public static boolean containsDuplicates(int[] arr) {
        Set<Integer> uniqueElements = new HashSet<>();

        for (int num : arr) {
            // If add() returns false, the element is already in the set
            if (!uniqueElements.add(num)) {
                return true; // Duplicate detected
            }
        }
        return false; // No duplicates found
    }
}
```

3. Find the first non-repeating element using HashMap.

Simple Explanation: We solve this in two passes. In the **first pass**, we use a HashMap to count the frequency of every element. In the **second pass**, we loop through the input again, and the first element whose count in the HashMap is exactly 1 is the answer.

```
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

public class NonRepeatingFinder {
    public static Character findFirstNonRepeatingChar(String str) {
        if (str == null || str.isEmpty()) {
            return null;
        }

        // Use LinkedHashMap to preserve insertion order (important for
        // finding the *first* non-repeating element)
        Map<Character, Integer> counts = new LinkedHashMap<>();

        // Pass 1: Count frequency of each character
        for (char c : str.toCharArray()) {
            counts.put(c, counts.getOrDefault(c, 0) + 1);
        }

        // Pass 2: Find the first character with a count of 1
        for (Map.Entry<Character, Integer> entry : counts.entrySet()) {
            if (entry.getValue() == 1) {
                return entry.getKey();
            }
        }
        return null; // All characters repeat
    }
}
```